

List of Experiment for LAB NLP TYCSE 2025 -26

Sr. No.	Title of the Experiments	Skill / Knowledge Level	CO	Marks for ISE
Level -all				
1.	Write a Python script to read data, Implement stopword removal using NLTK. Convert text to lowercase, remove punctuation, and handle contractions. Use regex to extract email IDs, phone numbers, or URLs from text.	SO1	CO1	3
2.	Write a Python script to tokenize a given text using NLTK and spaCy. Perform stemming and lemmatization.	SO2	CO1	3
3.	Write a Python script to Implement BoW and TF-IDF on a dataset using Scikit-learn.	SO2	CO2	2
4.	Write a Python script to implement an N-gram model for text generation :Extract unigrams, bigrams, and trigrams from a given text, Generate new text using an N-gram language model.	SO2	CO2	2
5.	Write a Python script to train a model (Naïve Bayes, SVM) to classify emails as spam or not spam.	SO3	CO3	2
6.	Write a Python script to Train Word2Vec embeddings using Gensim, Load pre-trained GloVe embeddings for text similarity analysis.	SO3	CO3	3
7.	Write a Python script to train a Naïve Bayes and Logistic Regression classifier for sentiment analysis.	SO3	CO3	3
8.	Write a Python script to implement LDA to discover hidden topics in a set of documents.	SO4	CO5	3
9.	Write a Python script to build a question-answering system using a fine-tuned BERT model.	SO4	CO5	3
10.	Write a Python script to implement text summarization using NLP techniques and transformers.	SO4	CO4	2

Experiment No. 1

LAB MANUAL

Experiment Title:

Text Preprocessing and Information Extraction using Python and NLTK

Aim

To write a Python script that:

- Reads textual data
- Implements stopword removal using NLTK
- Converts text to lowercase
- Removes punctuation
- Handles contractions
- Uses regular expressions (regex) to extract email IDs, phone numbers, and URLs from the text

Theory

Text preprocessing is a fundamental step in Natural Language Processing (NLP). Raw text data often contains noise such as punctuation, mixed casing, stopwords, and informal contractions, which can affect the performance of text analysis models.

1. Lowercasing:

Converting all text to lowercase ensures uniformity and avoids duplication of words (e.g., "Data" and "data").

2. Stopword Removal:

Stopwords are commonly used words such as "is", "the", "and", which carry minimal semantic meaning. Removing them reduces dimensionality and improves processing efficiency. NLTK (Natural Language Toolkit) provides a predefined list of stopwords.

3. Punctuation Removal:

Punctuation marks do not usually contribute to the meaning of text in most NLP tasks and are removed to clean the dataset.

4. Handling Contractions:

Contractions like "can't", "won't", "I'm" are expanded to "cannot", "will not", and "I am" to standardize the text.

5. Regular Expressions (Regex):

Regex is a powerful tool used for pattern matching. It is useful for extracting structured data such as:

- Email IDs
- Phone numbers
- URLs

Python Implementation (Write here your own code and Example)

```
import nltk
import re
import string
from nltk.corpus import stopwords

# Download stopwords (run once)
nltk.download('stopwords')

# Sample text
text = """Hello! My name is John Doe. Contact me at john.doe@example.com
or visit https://example.com. My phone number is +91-9876543210.
I can't attend the meeting today."""

# Step 1: Convert to lowercase
text_lower = text.lower()

# Step 2: Handle contractions
contractions = {
    "can't": "cannot",
    "won't": "will not",
    "i'm": "i am",
    "it's": "it is"
}

for key, value in contractions.items():
    text_lower = text_lower.replace(key, value)

# Step 3: Remove punctuation
text_no_punct = text_lower.translate(str.maketrans('', '', string.punctuation))

# Step 4: Remove stopwords
stop_words = set(stopwords.words('english'))
words = text_no_punct.split()
filtered_words = [word for word in words if word not in stop_words]

clean_text = " ".join(filtered_words)

# Step 5: Extract using regex
emails = re.findall(r'\S+@\S+', text)
phones = re.findall(r'\+?\d{8,12}\d', text)
urls = re.findall(r'https?://\S+|www\.\S+', text)

# Output
print("Cleaned Text:")
print(clean_text)

print("\nExtracted Emails:", emails)
print("Extracted Phone Numbers:", phones)
print("Extracted URLs:", urls)
```

Program and Output:

Conclusion

In this experiment, we successfully implemented a Python script for text preprocessing and information extraction. The script demonstrated essential NLP techniques such as stopwords removal, punctuation elimination, lowercase conversion, and contraction handling. Additionally, regex was effectively used to extract structured data like email addresses, phone numbers, and URLs. These preprocessing steps are crucial for improving the quality and efficiency of text analysis and machine learning models.

Experiment No. 2

Experiment Title: Tokenization, Stemming, and Lemmatization using NLTK and spaCy in Python

Aim:

To write a Python script that:

- Tokenizes a given text using **NLTK** and **spaCy**
- Performs **stemming** using NLTK
- Performs **lemmatization** using both NLTK and spaCy

Theory : Natural Language Processing (NLP) involves analyzing and processing human language, and an essential first step in this process is text preprocessing. Techniques such as tokenization, stemming, and lemmatization help convert raw text into a structured format suitable for analysis. Tokenization is the process of dividing text into smaller units called tokens, such as words, phrases, or sentences. Libraries like NLTK provide rule-based tokenization methods, while spaCy offers more advanced tokenization that handles punctuation, contractions, and special linguistic cases efficiently using pre-trained models.

1. Tokenization

Tokenization is the process of splitting text into smaller units called tokens (words, punctuation, or sentences). It is essential for converting unstructured text into a structured form.

- In **NLTK**, tokenization is rule-based and uses functions like `word_tokenize()`.
- In **spaCy**, tokenization is more advanced and handles punctuation, contractions, and special cases efficiently using pre-trained language models.

2. Stemming

Stemming is a technique used to reduce words to their root form by removing suffixes, without considering the actual meaning of the word. For example, words like “playing” and “played” may both be reduced to “play.” However, stemming can sometimes produce non-meaningful words, such as “happily” becoming “happi.” To overcome this limitation, lemmatization is used, which converts a word into its base or dictionary form, known as a lemma, by considering its context and grammatical role. For instance, “running” is converted to “run,” and “better” may be converted to “good.”

Example:

- *playing* → *play*
- *happiness* → *happy*

In NLTK, stemming is commonly done using the **Porter Stemmer**, which applies a set of heuristic rules.

3. Lemmatization

Lemmatization in NLTK is commonly performed using WordNet, a lexical database that organizes words into sets of synonyms called synsets and provides semantic relationships among them. The WordNet lemmatizer uses this database to return the base form of a word, but its accuracy improves significantly when the correct part-of-speech (POS) tag is provided; otherwise, it assumes the word to be a noun by default. On the other hand, spaCy performs lemmatization using built-in linguistic features such as vocabulary, morphological analysis, and automatic POS tagging. This allows spaCy to produce more accurate and context-aware lemmas without requiring manual input of grammatical information. Therefore, while NLTK with WordNet is simple and flexible for learning purposes, spaCy provides a more efficient and robust approach for real-world NLP applications.

Lemmatization converts a word to its base or dictionary form (lemma) using linguistic knowledge such as vocabulary and grammar. Unlike stemming, it produces meaningful words.

Example:

- *running* → *run*
- *better* → *good*

4. Lemmatization using WordNet (NLTK)

WordNet is a lexical database that groups English words into sets of synonyms (synsets) and provides semantic relationships.

- The **WordNet Lemmatizer** in NLTK uses this database to find the correct base form of a word.
- It works best when the **Part-of-Speech (POS)** tag is provided; otherwise, it assumes the word is a noun by default.

Example:

- *running (verb)* → *run*
- *better (adjective)* → *good*

Limitation: Without POS tagging, lemmatization may give less accurate results.

5. Lemmatization using spaCy

spaCy performs lemmatization using:

- Built-in vocabulary
- Morphological analysis

- POS tagging

Advantages of spaCy lemmatization:

- Automatically considers context and grammatical role
- More accurate than basic WordNet lemmatization
- No need to manually specify POS tags

Example:

- *are* → *be*
- *running* → *run*

Instructions for the Experiment

1. Select a short paragraph from any reliable news website (e.g., politics, sports, or technology news).
2. Copy the selected paragraph and use it as input text in your Python program.
3. Import the required libraries: **NLTK** and **spaCy**.
4. Perform **tokenization using NLTK** to split the paragraph into individual words.
5. Perform **tokenization using spaCy** and observe the differences in token handling.
6. Apply **stemming using NLTK (Porter Stemmer)** to reduce words to their root forms.
7. Perform **lemmatization using WordNet** via NLTK to obtain base forms of words.
8. Apply **lemmatization using spaCy** and compare the results with WordNet lemmatization.
9. Display all outputs clearly: tokens, stemmed words, and lemmatized words.
10. Compare and analyze the results obtained from NLTK and spaCy, noting differences in accuracy and output.

Program and Output:

Conclusion

Tokenization, stemming, and lemmatization are fundamental preprocessing steps in NLP. While NLTK provides simple and flexible tools (especially with WordNet), spaCy offers more advanced and context-aware processing, making it suitable for real-world applications.

Experiment No. 3

Experiment Title

Implementation of Bag of Words (BoW) and TF-IDF using Scikit-learn

Aim

To write a Python script to implement **Bag of Words (BoW)** and **TF-IDF** techniques on a given dataset using **Scikit-learn**.

Theory

Natural Language Processing (NLP) requires converting textual data into numerical form so that machine learning algorithms can process it. Two commonly used techniques for this purpose are Bag of Words (BoW) and Term Frequency–Inverse Document Frequency (TF-IDF). These methods are part of feature extraction in text processing.

The **Bag of Words (BoW)** model represents text as a collection of words, ignoring grammar and word order but keeping track of word frequency. In this approach, a vocabulary of all unique words in the dataset is created, and each document is represented as a vector indicating the frequency of each word. For example, if the dataset contains words like “data,” “science,” and “learning,” each document is converted into a vector based on how many times these words appear. Although simple and easy to implement, BoW does not capture semantic meaning or context.

The **TF-IDF (Term Frequency–Inverse Document Frequency)** technique improves upon BoW by assigning weights to words based on their importance. Term Frequency (TF) measures how often a word appears in a document, while Inverse Document Frequency (IDF) reduces the importance of commonly occurring words across all documents. Words that appear frequently in a specific document but rarely in others get higher weights, making TF-IDF more informative than simple frequency-based methods.

The formula for TF-IDF is:

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \log\left(\frac{1}{\text{DF}(t)}\right) \quad \text{TF-IDF}(t,d) = \text{TF}(t,d) \times \log\left(\frac{N}{\text{DF}(t)}\right)$$

where:

- $\text{TF}(t,d)$ = frequency of term t in document d
- N = total number of documents
- $\text{DF}(t)$ = number of documents containing term t

The implementation of BoW and TF-IDF is efficiently done using **Scikit-learn**, a popular Python library for machine learning. It provides:

- `CountVectorizer` for BoW representation
- `TfidfVectorizer` for TF-IDF representation

These tools automatically handle tokenization, vocabulary creation, and vectorization, making them highly efficient for large datasets.

Algorithm / Procedure

1. Import required libraries.
2. Define or load a text dataset (list of documents).
3. Initialize `CountVectorizer` for BoW.
4. Fit and transform the dataset to generate BoW matrix.
5. Initialize `TfidfVectorizer`.
6. Fit and transform the dataset to generate TF-IDF matrix.
7. Display feature names and corresponding vectors.
8. Compare BoW and TF-IDF outputs.

Program and Output:

Sample Output (Simplified)

BoW Feature Names:

```
['analysis' 'and' 'data' 'field' 'important' 'interdisciplinary'
 'is' 'learning' 'machine' 'of' 'part' 'science']
```

BoW Matrix:

```
[[0 0 1 1 0 1 1 0 0 0 0 1]
 [0 0 1 0 0 0 1 1 1 1 1 1]
 [1 1 1 0 1 0 0 1 1 0 0 0]]
```

TF-IDF Matrix:

```
[... weighted values ...],
 [...],
 [...]]
```

Conclusion

In this experiment, we implemented Bag of Words and TF-IDF techniques to convert textual data into numerical form using **Scikit-learn**. The BoW model provided a simple frequency-based representation, while TF-IDF improved the representation by assigning importance to words based on their relevance across documents. It was observed that TF-IDF gives more meaningful insights compared to BoW by reducing the weight of commonly occurring words. Hence, TF-IDF is generally preferred in real-world NLP applications for better text representation and improved model performance.

Experiment No. 4

Experiment Title

Implementation of N-gram Model for Text Generation in Python

Aim

To write a Python script to:

- Extract **unigrams, bigrams, and trigrams** from a given text
- Generate new text using an **N-gram language model**

Theory

In Natural Language Processing (NLP), an **N-gram model** is a probabilistic language model used to predict the next word in a sequence based on the previous $N-1$ words. It is one of the simplest and most fundamental approaches to text modeling and generation.

An **N-gram** is a contiguous sequence of N items (typically words) from a given text. Based on the value of N , different types of N-grams are defined:

- **Unigram (N=1):** Individual words
- **Bigram (N=2):** Pairs of consecutive words
- **Trigram (N=3):** Sequences of three consecutive words

For example, consider the sentence:

"Machine learning is powerful"

- Unigrams: *Machine, learning, is, powerful*
- Bigrams: *(Machine learning), (learning is), (is powerful)*
- Trigrams: *(Machine learning is), (learning is powerful)*

N-gram models work based on the concept of conditional probability. The probability of a word depends only on the previous $N-1$ words rather than the entire sentence. This assumption simplifies computation and is known as the **Markov assumption**.

For a bigram model, the probability is:

$$P(w_n | w_{n-1}) P(w_n \mid w_{\{n-1\}}) P(w_n | w_{n-1})$$

For a trigram model, the probability becomes:

$$P(w_n | w_{n-2}, w_{n-1}) P(w_n \mid w_{\{n-2\}}, w_{\{n-1\}}) P(w_n | w_{n-2}, w_{n-1})$$

Text generation using N-grams involves selecting the next word based on the probability distribution of possible words that follow a given sequence. Starting from an initial word or phrase, the model repeatedly predicts the next word to form a sentence.

Libraries like **NLTK** provide built-in support for generating N-grams and make it easier to implement such models efficiently.

Algorithm / Procedure

1. Import required libraries.
2. Input a sample text.
3. Preprocess the text (lowercase, tokenization).
4. Generate unigrams, bigrams, and trigrams.
5. Calculate frequency distribution of N-grams.
6. Build an N-gram model using probabilities.
7. Start with an initial word or phrase.
8. Predict the next word based on the model.
9. Repeat the process to generate new text.

Python Program (Write here your own program and output:)

```
import nltk
import random
from nltk.util import ngrams
from nltk.tokenize import word_tokenize
from collections import defaultdict, Counter

# Download tokenizer
nltk.download('punkt')

# Sample text
text = "Machine learning is a fascinating field of artificial intelligence and machine learning is powerful"

# Tokenization
tokens = word_tokenize(text.lower())

# Generate N-grams
unigrams = list(ngrams(tokens, 1))
bigrams = list(ngrams(tokens, 2))
trigrams = list(ngrams(tokens, 3))

print("Unigrams:\n", unigrams)
print("\nBigrams:\n", bigrams)
print("\nTrigrams:\n", trigrams)

# Build Bigram Model
```

```
bigram_model = defaultdict(list)
for w1, w2 in bigrams:
    bigram_model[w1].append(w2)

# Text Generation Function
def generate_text(model, start_word, length=10):
    word = start_word
    result = [word]

    for _ in range(length):
        next_words = model.get(word)
        if not next_words:
            break
        word = random.choice(next_words)
        result.append(word)

    return ' '.join(result)

# Generate text
generated = generate_text(bigram_model, start_word="machine", length=10)
print("\nGenerated Text:\n", generated)
```

Sample Output

Unigrams:

[('machine',), ('learning',), ('is',), ('a',), ...]

Bigrams:

[('machine', 'learning'), ('learning', 'is'), ...]

Trigrams:

[('machine', 'learning', 'is'), ...]

Generated Text:

machine learning is powerful and machine learning is fascinating field

Conclusion

In this experiment, an N-gram language model was implemented to understand and generate text. Unigrams, bigrams, and trigrams were extracted to analyze word sequences, and a simple probabilistic model was used for text generation. It was observed that N-gram models are easy to implement and useful for basic language modeling tasks. However, they have limitations such as lack of long-term context understanding and data sparsity. Despite these limitations, N-gram models form the foundation for more advanced NLP techniques and are useful for educational and introductory purposes.